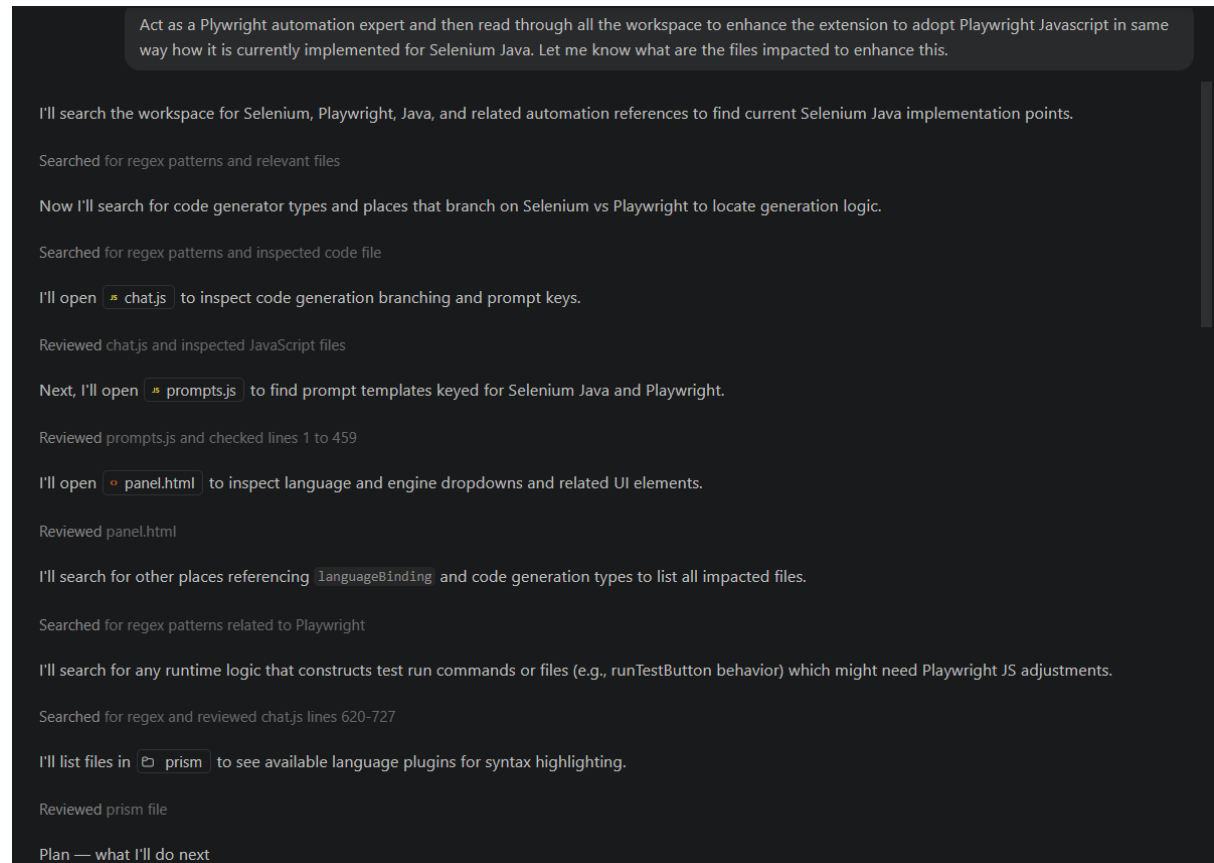


Note: Since the assignment is to enhance the extension with Playwright-typescript but the implementation is already available in the tool, I am going with Playwright/javascript.

GitHub Copilot in VSCode with Prompt in ASK mode



Progress update

- I scanned the UI, generator logic and prompt templates in the workspace (key files: panel.html, src/scripts/chat.js, src/scripts/prompts.js, lib/prism).

Files impacted and required changes

- `panel.html:97-116` : add a JavaScript option to the `languageBinding` select (value `js` or `javascript`) and include a Prism JavaScript plugin script (e.g., `lib/prism/prism-javascript.js`) for highlighting.
- `chatjs:1-120` and `chatjs:360-580` : add a helper `isJavaScriptPlaywright(language, engine)` (detect `js / javascript` + `playwright`), update `getPromptKeys()` to map JS+Playwright to new `PLAYWRIGHT_JAVASCRIPT_*` prompt keys (page-only, spec-only, and combined), and adjust any user-facing unsupported-language messages to include Playwright JS support.
- `prompts.js:1-120` and `prompts.js:400-800` : add new prompt templates:
 - `PLAYWRIGHT_JAVASCRIPT_POM` (Playwright Page Object in JavaScript, async methods, Playwright locators)
 - `PLAYWRIGHT_JAVASCRIPT_SPEC` (Playwright test spec in JavaScript using `@playwright/test`)
 - `PLAYWRIGHT_JAVASCRIPT_POM_AND_SPEC` (both POM and spec)Also add these keys to `CODE_GENERATOR_TYPES`.
- `lib/prism/prism-javascript.js` (new file): add or copy the Prism JavaScript language plugin and include it from `panel.html` so generated JS is highlighted correctly.
- (Optional) Update any references or docs: if there's a README or stored default settings that enumerate supported language values, add `js / javascript` there as well.

Why each change is needed (brief)

- UI: users must be able to pick JavaScript as the binding.
- chat.js: generation branching currently only recognizes `ts` + Playwright; a JS detector and prompt-key mapping are required.
- prompts.js: currently contains Playwright TypeScript prompts — equivalent JavaScript prompts must be supplied so the LLM produces idiomatic JS Playwright code.
- Prism: add JS syntax support so rendered code blocks highlight properly.

Impacted Files

- UI: `panel.html:97-116` — add JavaScript option to the `languageBinding` select and include Prism JavaScript plugin script.
- Generator logic (selection + tokens + UI state): `chatjs:1-120` — language/browser element bindings and `codeGeneratorType` init.
- Prompt key mapping & mode logic: `chatjs:360-420` — `getPromptKeys()` and selection handling; add `isJavaScriptPlaywright()` helper and map JS+Playwright to new prompt keys.
- Prompt templates (add JS Playwright templates): `prompts.js:1-200` and `prompts.js:200-480` — add:
 - `PLAYWRIGHT_JAVASCRIPT_POM`
 - `PLAYWRIGHT_JAVASCRIPT_SPEC`
 - `PLAYWRIGHT_JAVASCRIPT_POM_AND_SPEC`
- Generator type registry: `prompts.js:760-800` — add the above keys to `CODE_GENERATOR_TYPES`.
- Syntax highlighting (new file to add): `lib/prism/prism-javascript.js` — add/copy Prism JavaScript language plugin; include it from `panel.html`.
- App config / initialization (verify defaults): `init-config.js:1-40` — (verify if default language/codeGeneratorType needs updating).
- Optional docs or settings: any README or settings docs (not present as required) — update to include `js / javascript` Playwright support.

Ask a question about your code or project

+ ⓘ Ask Auto ↺

Local Default Approvals

Switching to **Agentic mode** with below prompt

Please accomodate the changes in the workspace to enhance the tool with Playwright Javascript. Ensure that the impacted files only modified to get the changes based on above mentioned files. Once each impacted files modification is completed provide a message to user as a notification.

Optimized tool selection

I'm implementing the Playwright JavaScript support in the workspace now and will report each impacted file as it's updated.

Considered JavaScript plugin options and evaluated syntax highlighting

I've reached the implementation step and am wiring the new JavaScript Playwright path through the UI, prompt mapping, and prompt templates now.

Updated panel.html +231 -1

The core support is in place. I'm adding the missing Prism JavaScript include file and then I'll verify the workspace for syntax issues.

Created prism-javascript.js and checked 3 files

The change set is validated and I'm giving you a concise completion summary with the files updated.

Implemented Playwright JavaScript support in the workspace.

Notifications by file

- Updated `panel.html` — added JavaScript as a supported language option and loaded the JavaScript Prism plugin.
- Updated `chat.js` — added Playwright JavaScript detection and routing to new prompt keys for page-object/spec/combined generation.
- Updated `prompts.js` — added Playwright JavaScript prompt templates for POM, spec, and combined generation, plus registry entries.
- Added `prism-javascript.js` — added Prism support for JavaScript syntax highlighting.

Verification

- I checked the edited files with the workspace error checker and found no errors in `chat.js`, `prompts.js`, or `panel.html`.

What this enables

- You can now choose JavaScript with Playwright from the extension UI.
- The generator will produce Playwright JavaScript code for:
 - Page Object Model
 - Test spec
 - Combined POM + spec

If you want, I can next help you test the extension flow in the browser and validate the generated output.

MAI-Code-1-Flash • 8.8 credits

> 4 files changed +232 -2

Keep Undo

+ JS `prompts.js`

Describe what to build

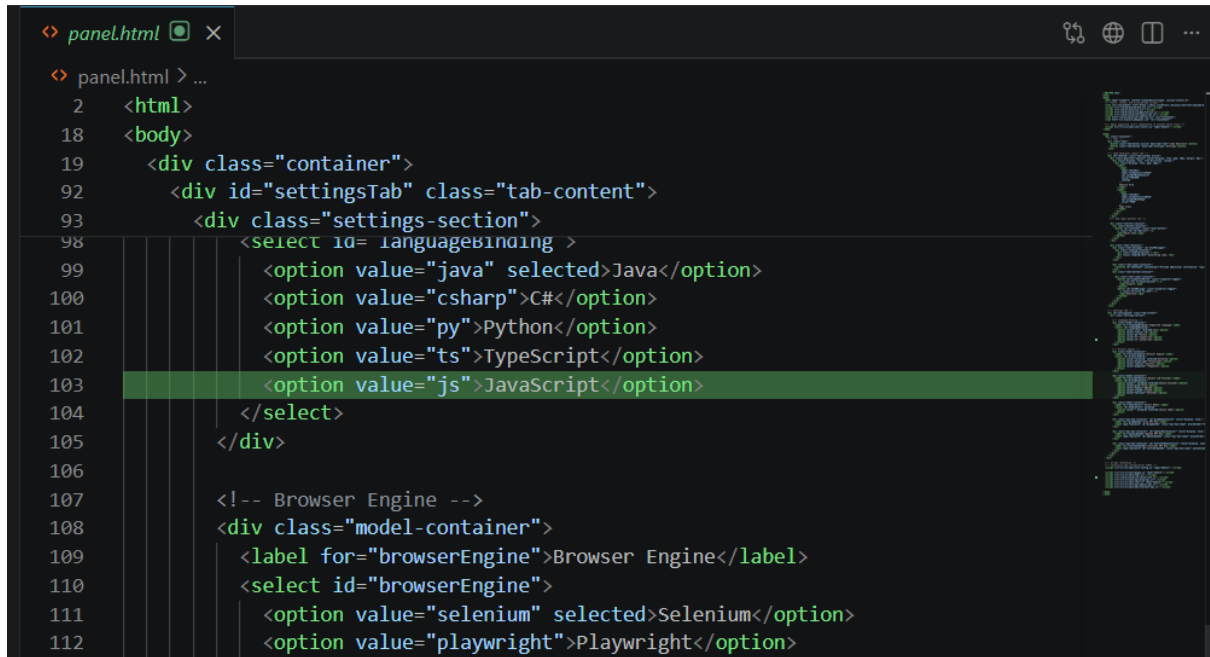
+ | Agent | Auto |

Local | Default Approvals

Prompt.js updated with a new section for for POM, spec, and combined generation.

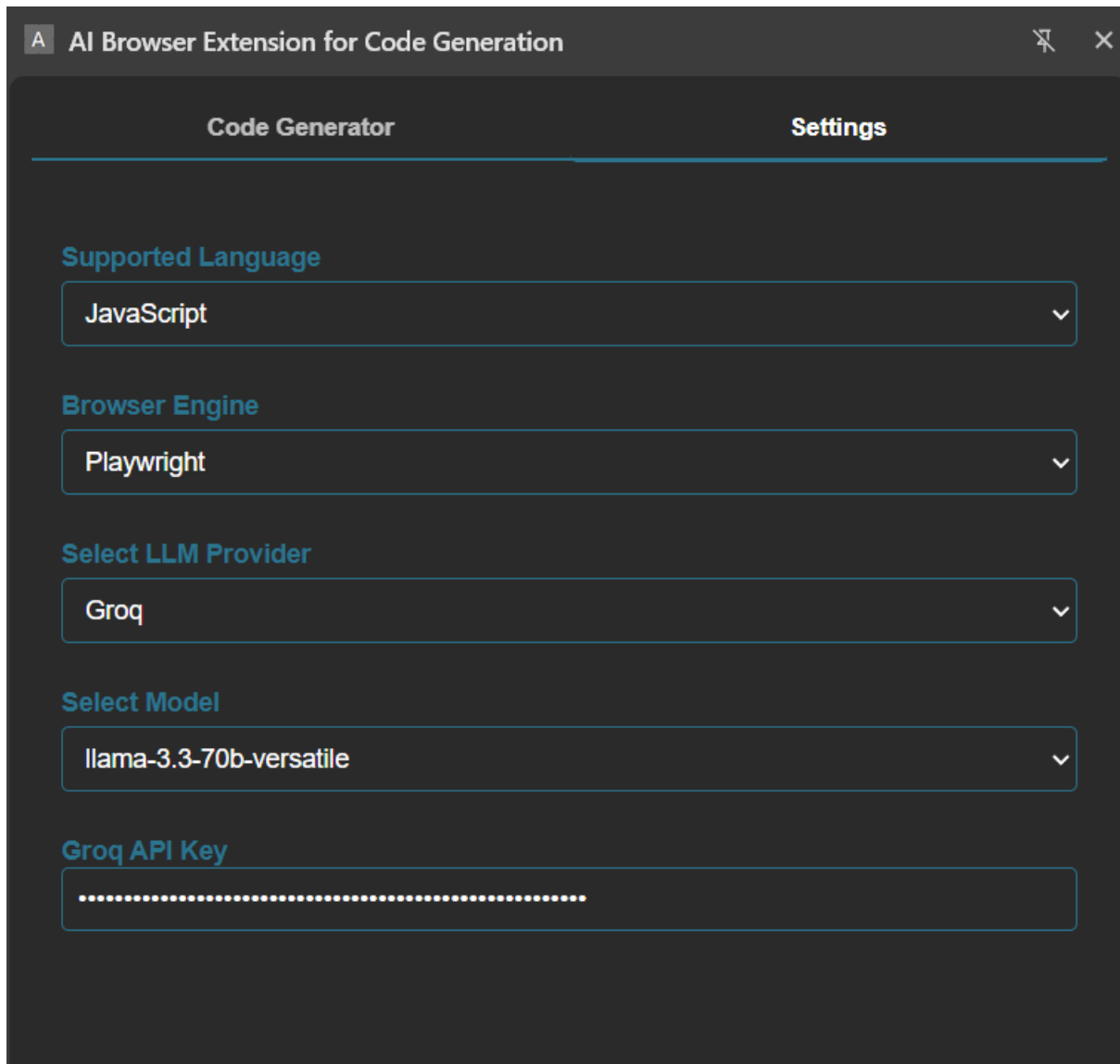
```
JS prompts.js X
src > scripts > JS prompts.js > [6] DEFAULT_PROMPTS > [6] PLAYWRIGHT_JAVASCRIPT_POM
4 export const DEFAULT_PROMPTS = {
94 CUCUMBER_WITH_SELENIUM_JAVA_STEPS: `
186 Tone:
187 - Professional, executable, structured.
188 `
189
190 /**
191  * Playwright JavaScript Page Object Model (No Test)
192  */
193 PLAYWRIGHT_JAVASCRIPT_POM: `
194 Instructions:
195 - Generate ONLY a Playwright JavaScript Page Object Model class (no test code).
196 - Add JSDoc comments for all methods and the class.
197 - Use Playwright locators (page.locator(), page.getByRole(), page.getByLabel(),
198 - Make all methods async and use async/await syntax.
199 - Use meaningful method names following camelCase convention.
200 - Do NOT include test code or fixture setup.
201 - Do NOT include test file runner logic.
202
203 Context:
204 DOM:
205 \`\`\`html
206 \`${domContent}`
207 \`\`\`
208
209 Example:
210 \`\`\`javascript
211 const { Page, Locator } = require('@playwright/test');
212
213 /**
214  * Page Object Model for Login Page
215  */
216 class LoginPage {
217   constructor(page) {
218     this.page = page;
219     this.usernameInput = page.locator('input[id="username"]');
220     this.passwordInput = page.locator('input[id="password"]');
221     this.loginButton = page.locator('button:has-text("Login")');
222   }
223
224   async goto() {
225     await this.page.goto('/login');
```

Panel.html->Newly added Javascript as the option



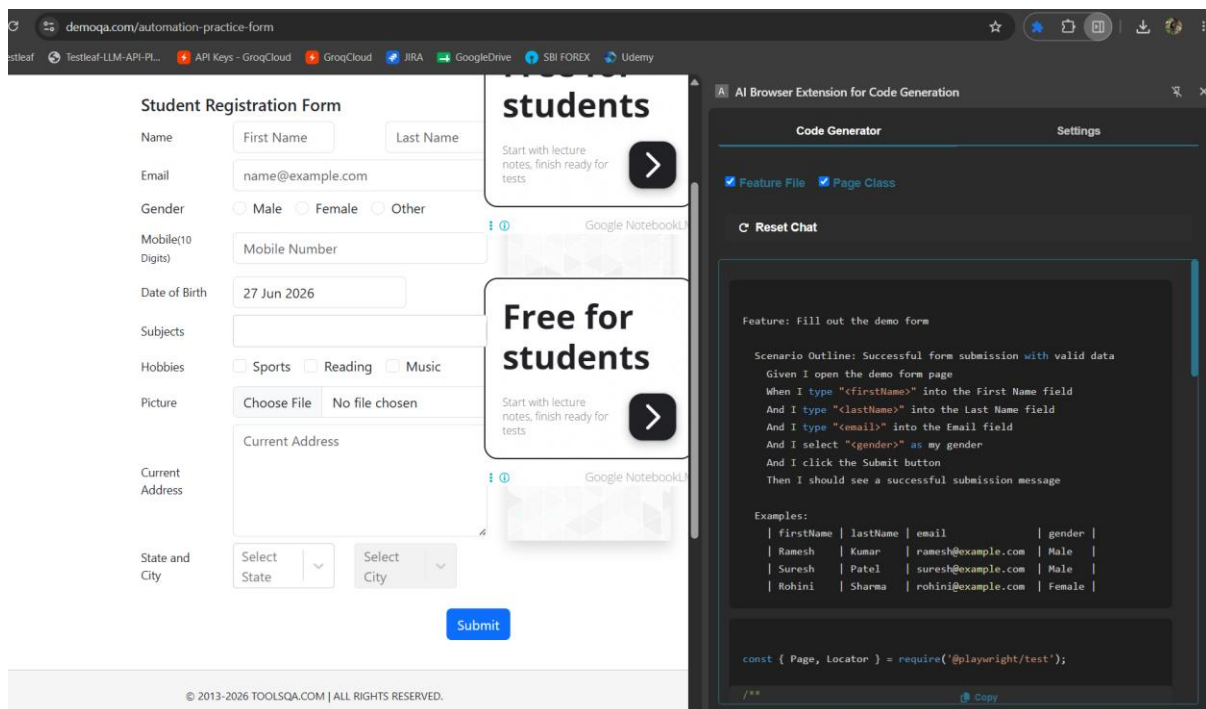
```
<? panel.html > ...
2  <html>
18  <body>
19    <div class="container">
92      <div id="settingsTab" class="tab-content">
93        <div class="settings-section">
98          <select id= languagebinding >
99            <option value="java" selected>Java</option>
100            <option value="csharp">C#</option>
101            <option value="py">Python</option>
102            <option value="ts">TypeScript</option>
103            <option value="js">JavaScript</option>
104          </select>
105        </div>
106
107        <!-- Browser Engine -->
108        <div class="model-container">
109          <label for="browserEngine">Browser Engine</label>
110          <select id="browserEngine">
111            <option value="selenium" selected>Selenium</option>
112            <option value="playwright">Playwright</option>
```

Tool selected for Playwright +Javascript



Working as expected:

Feature file:



Feature: Fill out the demo form

Scenario Outline: Successful form submission with valid data

Given I open the demo form page

When I type "<firstName>" into the First Name field

And I type "<lastName>" into the Last Name field

And I type "<email>" into the Email field

And I select "<gender>" as my gender

And I click the Submit button

Then I should see a successful submission message

Examples:

firstName	lastName	email	gender
-----------	----------	-------	--------

Ramesh	Kumar	ramesh@example.com	Male
--------	-------	--------------------	------

Suresh	Patel	suresh@example.com	Male
--------	-------	--------------------	------

| Rohini | Sharma | rohini@example.com | Female |

Page Class

```
const { Page, Locator } = require('@playwright/test');

/**
 * Page Object Model for the demo form page.
 */
class DemoFormPage {
  /**
   * Creates an instance of DemoFormPage.
   * @param {Page} page - The Playwright page object.
   */
  constructor(page) {
    this.page = page;

    this.firstNameInput = page.locator('input[id="firstName"]');
    this.lastNameInput = page.locator('input[id="lastName"]');
    this.emailInput = page.locator('input[id="userEmail"]');
    this.maleRadio = page.locator('input[id="gender-radio-1"]');
    this.femaleRadio = page.locator('input[id="gender-radio-2"]');
    this.submitButton = page.locator('button:has-text("Submit")');
  }

  /**
   * Navigates to the demo form page.
   */
}
```



```
async goto() {  
    await this.page.goto('https://demoqa.com/automation-practice-form');  
}
```

```
/**
```

```
 * Enters the first name into the corresponding field.
```

```
 * @param {string} firstName - The first name to enter.
```

```
 */
```

```
async enterFirstName(firstName) {  
    await this.firstNameInput.fill(firstName);  
}
```

```
/**
```

```
 * Enters the last name into the corresponding field.
```

```
 * @param {string} lastName - The last name to enter.
```

```
 */
```

```
async enterLastName(lastName) {  
    await this.lastNameInput.fill(lastName);  
}
```

```
/**
```

```
 * Enters the email into the corresponding field.
```

```
 * @param {string} email - The email to enter.
```

```
 */
```

```
async enterEmail(email) {  
    await this.emailInput.fill(email);  
}
```

```

/**
 * Selects the gender.
 * @param {string} gender - The gender to select (Male or Female).
 */
async selectGender(gender) {
  if (gender === 'Male') {
    await this.maleRadio.check();
  } else if (gender === 'Female') {
    await this.femaleRadio.check();
  }
}

/**
 * Clicks the Submit button.
 */
async clickSubmit() {
  await this.submitButton.click();
}

}

module.exports = { DemoFormPage };

```